

Authentication & Session Development Contract

Owner: A

Dependency order: Wave 1. A merges the public entry point and declarations first; Application Shell, Account Privacy, and Account Center then consume them in parallel.

Definition of done: Consumers can distinguish real sessions, confirmation status, and failures from this document alone, and can safely coordinate sign-in, sign-out, and account switching.

This is the sole cross-owner development contract for Authentication & Session and the Markdown source for the human-readable PDF. It fixes public Dart declarations, result semantics, data boundaries, and joint acceptance. Widgets, state management, Supabase SDK mapping, retries, private files, and test organization within `lib/features/authentication_session/` remain the Owner's choice.

1. Outcome, responsibilities, and dependencies

- A user can sign in to the current device with email and password; failures provide an understandable correction or retry path.
- A user can register; the UI distinguishes an established session, a requirement to check verification email, and a failure that did not establish a session. Profile-registration failure does not negate authentication.
- The account page presents only Auth's real email and confirmation state; an unknown state never displays “verified”.
- Sign-out ends only the current-device session. When switching accounts, old-account private content is blocked and cleared first, without affecting other devices or remote business records.

Owner	Responsible for	Not responsible for	Dependency
Authentication & Session (A)	Email/password authentication, current-device session, true email/confirmation facts, optional registration profile, sole public entry point	Application gating/navigation, private-payload cleanup, account business pages, other-device sessions	Encapsulates Supabase Auth and direct profiles access
Application Shell	Authentication gate, auth-entry composition, sign-out/switch orchestration, and private-UI barrier	Auth SDK and authentication rules	Consumes AUTH-001; calls PRIVACY-001.open after authentication
Account Privacy	Account-scope opened/closed facts and local cleanup	Authentication and navigation	Consumes the AUTH-001 account id; Shell is its only initiator
Account Center	Presents real email/confirmation state and sign-out intent	Direct Auth or profiles access	Consumes AUTH-001; sends sign-out intent to Shell

Scope includes sign-in, registration, true email confirmation, session restoration/end, optional registration profile, and the authentication side of ACCOUNT-07. It excludes Magic Link, guests, password reset, fixed “Verified User” copy, and Account Privacy cleanup implementation. Product facts: [authentication](#), [account](#). System flow: [FLOW-01](#).

2. Interfaces this Owner must call

Supabase Auth seam (AUTH-002, A only)

A is the only Owner that calls Supabase Auth directly. SDK methods, token refresh, cancellation, retries, and error mapping are internal implementation; other Owners must not depend on them. Only an **explicitly valid or successfully restored** session can become an authentication fact. Expired cache, refresh-in-progress, refresh failure, offline non-confirmability, and remote rejection must not open a private scope.

Directly accessed object	Purpose and invariant
auth.users	Supplies account id, real email, email-confirmed state, and current-device session. Credentials/tokens are not copied to public data, and email/confirmation are never inferred from profiles.
profiles	Written only after authentication for optional username/avatar/bio; id equals the current Auth account id and CRUD is owner-only. Failure returns a profile result and never revokes or fabricates authentication.

The [Schema Catalog](#) is the sole authority for fields, RLS, and migrations.

3. Interfaces this Owner must provide

Current-device session (AUTH-001)

Provider: Authentication & Session (A)

Consumers: Application Shell, Account Privacy, Account Center

Only public import: `package:locatemy/features/authentication_session/authentication_session.dart`

Consumers may import this entry point only, never `lib/features/authentication_session/src/`. A first submits the following **declarations** and minimum public types; A writes all function bodies.

```

abstract interface class AuthenticationSession {
    Future<SessionSnapshot> restoreSession();
    Stream<SessionSnapshot> watchSession();
    Future<SignInOutcome> signIn({
        required String email, required String password,
    });
    Future<RegistrationOutcome> register({
        required String email,
        required String password,
        required String passwordConfirmation,
        String? username,
    });
    Future<SignOutOutcome> signOut();
}

sealed class SessionSnapshot {}
final class AuthenticatedSession extends SessionSnapshot {
    final AuthenticatedAccount account;
}
final class UnauthenticatedSession extends SessionSnapshot {}
final class SessionUnavailable extends SessionSnapshot {
    final SessionFailure failure;
}

final class AuthenticatedAccount {
    final String accountId;
    final String email;
    final EmailConfirmation confirmation;
}
enum EmailConfirmation { confirmed, verificationRequired, unavailable }

sealed class SignInOutcome {}
final class SignInSucceeded extends SignInOutcome {
    final AuthenticatedAccount account;
}
final class SignInRejected extends SignInOutcome {
    final SignInFailure failure;
}

sealed class RegistrationOutcome {}
final class RegistrationAuthenticated extends RegistrationOutcome {
    final AuthenticatedAccount account;
    final ProfileRegistrationOutcome profile;
}
final class RegistrationVerificationRequired extends RegistrationOutcome {
    final String email;
    final ProfileRegistrationOutcome profile;
}
final class RegistrationRejected extends RegistrationOutcome {
    final RegistrationFailure failure;
}

sealed class ProfileRegistrationOutcome {}
final class ProfileRegistered extends ProfileRegistrationOutcome {}
final class ProfileRegistrationSkipped extends ProfileRegistrationOutcome {}
final class ProfileRegistrationFailed extends ProfileRegistrationOutcome {
    final ProfileFailure failure;
}

sealed class SignOutOutcome {}
final class SignOutSucceeded extends SignOutOutcome {}
final class SignOutRejected extends SignOutOutcome {
    final SignOutFailure failure;
}

enum SessionFailure { retryableUnavailable, unsupportedClient, remoteRejected }
enum SignInFailure { invalidInput, invalidCredentials, retryableUnavailable, unsupportedClient }
enum RegistrationFailure { invalidInput, accountAlreadyExists, retryableUnavailable, unsupportedClient }
enum ProfileFailure { retryableUnavailable, permissionDenied, unknown }
enum SignOutFailure { retryableUnavailable, remoteRejected, unsupportedClient }

```

This is the collaboration shape, not a prescription for sealed-class file layout, SDK enums, or error copy. New public members or result variants require the agreement process in section 7.

Inputs

Call	Inputs and constraints	Caller responsibility
restoreSession / watchSession	None	Shell calls only while the authentication gate has not opened private routes; every snapshot re-decides whether a scope can open.
signIn	Non-empty email, password	Password exists only for this form/call; never log, persist, or send it to another Owner.
register	Non-empty email, password, passwordConfirmation; nullable username	Confirmation must match password before submission; absent username skips profile registration. Password strength and copy remain A's choice within product facts.
signOut	None	Only Shell calls it after old private UI is blocked; Account Center never calls it directly.

Outputs, failures, and caller handling

Result	Meaning	Required consumer handling
AuthenticatedSession, SignInSucceeded, RegistrationAuthenticated	The device has a clearly valid session; account is immutable Auth identity and real email fact	Shell calls PRIVACY-001.open with the same accountId and creates private UI only after opened; Account Center uses confirmation.
UnauthenticatedSession	No usable session is known	Display auth entry; never private UI.
SessionUnavailable	The session cannot safely be confirmed; failure distinguishes retryable network, unsupported client, and remote rejection	Stay gated with a recovery path; do not reduce to unauthenticated or open private UI.
RegistrationVerificationRequired	Registration requires checking verification email; email is the registration target	Show check-email state, never “verified”; if a clear session later exists, scope still follows SessionSnapshot.
confirmed / verificationRequired / unavailable	Auth-confirmed, explicitly pending, or no confirmation fact available	Present verified, pending verification, or unavailable respectively; never guess unknown.
Any *Rejected	This action has no corresponding success semantics	Use its enum for correction, retry, or escalation; do not parse strings or create scope.
ProfileRegistrationFailed	Authentication/verification result remains valid; only profile write failed	Show profile retry, never describe it as sign-in/registration failure.

The first and subsequent watchSession snapshots have identical semantics; a historic authenticated value cannot authorize continued private display.

Side effects, permissions, and order

- accountId comes exclusively from Auth; public types expose no password, token, SDK payload, or profiles field.
- Sign-in, registration, and restoration report authentication facts only: they do not navigate, open scope, or clear another module's data.
- signOut ends the **current-device** session only; it neither deletes remote records nor ends other-device sessions.
- Shell's fixed sign-out/switch order is: block old private UI and new business intent → signOut → PRIVACY-001.close(oldAccountId) → display ordinary sign-in only after both succeed. Any failure remains private-content-free with recovery.
- If an opened scope's account id differs from AuthenticatedSession, Shell displays no private content for either account; it closes the old scope under FLOW-01 and reauthenticates.

Minimal invocation (Shell)

```
final snapshot = await authenticationSession.restoreSession();
switch (snapshot) {
  case AuthenticatedSession(:final account):
    // Call PRIVACY-001.open with account.accountId; enter the app only after opened.
  case UnauthenticatedSession():
    // Show sign-in/registration entry.
  case SessionUnavailable(:final failure):
    // Remain gated and provide recovery for failure.
}
```

This illustrates seam usage, not a submit-ready Shell implementation.

Fake Adapter scenario

Shell uses a fake AuthenticationSession returning AuthenticatedSession(account A), UnauthenticatedSession, and SessionUnavailable(retryableUnavailable) to prove it creates private UI only after A's PRIVACY-001.open returns opened. Account Center uses the same fake with all three EmailConfirmation values to prove unavailable is never presented as verified. Neither needs Supabase or A's production implementation.

4. Recommended implementation order

- 1. **A: merge the public seam first.** Create the sole entry point, declarations, and minimum fake; consumers must not depend on A internals before merge.
- 2. **A: implement the real Adapter and forms.** Map Supabase Auth/profiles to these results and write real-Adapter contract tests.
- 3. **Consumers: work in parallel with the fake.** Shell implements gating and FLOW-01; Account Privacy receives only account id; Account Center uses only AuthenticatedAccount.
- 4. **Integrate together.** Wire the production seam and retain only the necessary cross-module flow tests in section 5.

5. Joint acceptance

Scenario	Owners	Action	Observable result	Trace
Restore at cold start	A, Shell, Privacy	No session, valid A session, non-confirmable session	Auth entry; main app only after scope opened; or recovery gate without private content	AUTH-01; AT-AUTH-01
Bad credentials or network	A, Shell	Invalid/bad credentials, offline, unsupported client	Distinct failure/recovery, no scope, no password leak	AUTH-01; AT-AUTH-01, AT-OUT-02
Registration and real confirmation	A, Shell, Account Center	authenticated, verification required, profile failure	Separate authentication/verification/profile states; only Auth fact can show verified	AUTH-02, AUTH-03; AT-AUTH-02
Current-device sign-out	A, Shell, Privacy	Confirm sign-out; inject Auth/cleanup failure	Ordinary sign-in after success; failure has no private UI and can retry; remote/other devices unchanged	ACCOUNT-07; AT-OUT-01, AT-OUT-02
Switch A to B	A, Shell, Privacy, private Owners	A scope open, then sign out and sign in as B	A data unreadable before B scope opens; only B data then appears	ACCOUNT-07; AT-SWITCH-01

6. Implementation freedom, blockers, and references

The Owner may choose src/ structure, widgets, state management, SDK Adapter, refresh/retry/cancellation, local validation, and test organization. Pause for agreement if the sole public entry point, public declaration/result/order/permission changes; profiles schema/RLS must change; or a Supabase version invalidates the locked conclusion of RISK-SESSION-01.

Current blockers: none. Reopen RISK-SESSION-01 if dependency versions change. Authorities: [Schema Catalog](#), [Interface registry](#), [session risk](#).

7. Contract changes and completion checks

A public Interface change requires the provider to explain reason and affected consumers, and every affected consumer to confirm it. The same PR updates the public declarations, this contract, affected fake/Adapter tests, and the PDF. Git/PR history is authoritative; document versions, checksums, manifests, Generation Gates, and Development Releases are not used.

- [x] Owners, consumers, dependency order, and sole public entry point are clear.
- [x] AUTH-001 declarations, inputs, results, failures, side effects, permissions, ordering, example, and fake scenario are complete.
- [x] Direct data objects and permissions have one authority; consumers do not receive persistence schema.
- [x] Joint acceptance covers restoration/gating, registration/verification, sign-out/cleanup, and A→B switching.